UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Department of Computer Science

COS 132 - Imperative Programming
Study Unit 8: Advanced Types in C++

Contents

8.1 Introduction

On completion of this study unit the student should:

- Understand what an array (one- or multi-dimensional) is.
- Write programs that manipulate arrays in C++.
- Be able to search for elements in an array.
- Be able to sort array elements.

8.2 One-Dimensional (Static) Arrays

A mechanism is required to store a number of values of the same type for later use. An array makes it possible to associate one variable name with a number of values of the same type. The individual values are addressed or retrieved by using an index or pointer.

Example: The name and two test marks for 200 students are typed in. Each name, the two marks, the average of the two marks and finally a class average is output. The program is given by:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    float mark1, mark2, sumAverage = 0.0, average;
    int count;
    string name;

    for (count = 1; count <= 200; ++count) {
        cout << "Enter student name: ";
        cin >> name;
        cout << "Enter two marks, separated by a space: ";
        cin >> mark1 >> mark2;
        average = (mark1 + mark2) / 2.0;
        cout << name << " " << mark1 << " " << mark2 << " " << average << endl;
        sumAverage += average;
    }

    cout << "The average is " << sumAverage / 200.0 << "%" << endl;

    return 0;
}
```

Listing 8.1: Basic Average Calculation

Suppose the problem statement further specifies that if the class average is less than 50 the student's marks should be increased by 5. The StudentMarks program isn't suitable

because by the time the class average is calculated - the student's averages aren't available any more. Only the last student's average will still be in the variable average. Must we then read in all the marks again? This does not seem to be the answer to our problem.

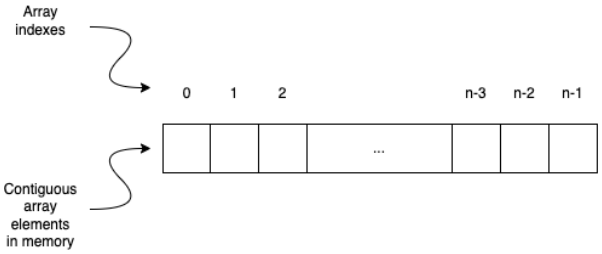
To solve this problem at a first glance, we need 200 variables to store each student's average. To increase the averages implies 200 assignment statements:

```
#include <iostream>
#include <string>
using namespace std;

void calculateCorrectAverage(int count, float mark1, float mark2, ..., float mark200,
        float &sumAverage, float &average1, float &average2, ..., float &average200) {
    switch (count) {
        case 1:
            average1 = (mark1 + mark2) / 2.0;
            cout << average1 << endl;
            sumAverage += average1;
            break;
        case 2:
            average2 = (mark1 + mark2) / 2.0;
            cout << average2 << endl;
            sumAverage += average2;
            break;
        // Add cases for other students as needed
        case 200:
            average200 = (mark1 + mark2) / 2.0;
            cout << average200 << endl;
            sumAverage += average200;
            break;
    }
}

void readAll(float &sumAverage, float &average1, float &average2, float &average200) {
    float mark1, mark2;
    string name;
    for (int count = 1; count <= 200; ++count) {
        cout << "Enter student name: ";
        cin >> name;
        cout << "Enter two marks, separated by a space: ";
        cin >> mark1 >> mark2;
        cout << name << " " << mark1 << " " << mark2 << " ";
        calculateCorrectAverage(count, mark1, mark2, sumAverage, average1, average2, average200);
    }
}

void adjustAverages(float &average1, float &average2, ..., float &average200) {
    average1 += 5;
    average2 += 5;
    // Add adjustments for other students as needed
```

```
    average200 += 5;
}

int main() {
    float sumAverage = 0.0;
    float average1, average2, average200;

    readAll(sumAverage, average1, average2, average200);
    sumAverage /= 200.0;

    if (sumAverage < 50.0) {
        adjustAverages(average1, average2, average200);
        sumAverage = (average1 + average2 + ... + average200) / 200.0;
    }

    cout << "Adjusted average is " << sumAverage << "%" << endl;

    return 0;
}
```

Listing 8.2: Adjust mark without for loop

It should be clear that the solution given in Listing 8.2 is a totally inefficient way to solve a problem. An array should be used for problems of this kind where multiple values of the same type are to be stored in memory. A variable of type array has one name and it can contain many values of the same type. We can think of an array as a group of boxes that are joined together, numbered 0, 1, 2,... etc. Each value of an array can be retrieved by its position in the array. One variable is therefore associated with many values and an index is used to identify a specific value. The array is a structured datatype that exists in most programming languages. Visually an array can be represented in the following manner:



The elements of the type array have a linear structure - each value has a unique predecessor and successor, except the first and last values. The size of the array (an integer value, say `n`) should be given at array declaration. The indexes of the array then range from `0` to `n-1`.

The syntax for an array of type *typeName* is as follows:

```
const int numberOf = 100;
typeName arrName[numberOf];
//eg. For an int array
int arrName[numberOf];
```

```
// OR
int arrName[100];
```

Listing 8.3: Array Declaration Syntax

Note, the `numberOf` field within square brackets [], representing the number of elements in the array, must be a constant expression, since arrays are blocks of static memory whose size must be determined at compile time, before the program runs. In other words you cannot change the size of the array at runtime, eg. though user input.

In the above case all the elements in `arrName` are still uninitialised after declaration. The elements in an array can be explicitly initialised to specific values when it is declared, by adding those initial values in braces . For example:

```
int foo [5] = { 16, 2, 77, 40, 12071 };
//Or:
int foo2 [] = { 16, 2, 77, 40, 12071 }; //Since 5 values are provided, size is 5
```

Listing 8.4: Array Initialisation

Accessing the elements of an array is done by specifying the name of the array with the index in square brackets, that is: `arrName[index]`, for example, the 10^{th} `int` in the array called `arrName`:

```
int index = 9; // remember array indexing starts from 0
int arrName[index];
```

Listing 8.5: Array Access Syntax

If we were to execute the following statement `cout << arrName endl;`, we would receive a reference (memory address) on the first element of the array (or equivalent to `&matrix[0]`).

The marks program is now written as follows using arrays:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    float mark1, mark2, sumAverage = 0.0;
    float average[200];
    int count;
    string name;

    for (count = 0; count < 200; ++count) {
        cout << "Enter student name: ";
        cin >> name;
        cout << "Enter two marks, separated by a space: ";
        cin >> mark1 >> mark2;
        average[count] = (mark1 + mark2) / 2.0;
        cout << name << " " << mark1 << " " << mark2 << " " << average[count] << endl;
        sumAverage += average[count];
    }
```

```
    sumAverage /= 200.0;
    if (sumAverage < 50.0) {
        sumAverage = 0.0;
        for (count = 0; count < 200; ++count) {
            average[count] += 5;
            sumAverage += average[count];
        }
        sumAverage /= 200.0;
    }

    cout << "The total average is " << sumAverage << "%" << endl;

    return 0;
}
```

Question 8.1
Why must `sumAverage` be re-assigned the value `0.0` in the main program?

Question 8.2
Extend the program to include an array of student names. Note, this question will be expanded on in Question 8.14

Consider the following example that counts the number of times each number on a dice occurred after each subsequent roll of the dice. The dice was thrown an unknown number of times. The program without arrays would be written as follows:

```
#include <iostream>
using namespace std;

int main() {
    int counter1 = 0, counter2 = 0, counter3 = 0;
    int counter4 = 0, counter5 = 0, counter6 = 0;
    int throwValue;
    int total = 0;

    cout << "Number on dice with first throw: ";
    cin >> throwValue;

    while (throwValue != 0) {
        switch (throwValue) {
            case 1: counter1++; break;
            case 2: counter2++; break;
            case 3: counter3++; break;
            case 4: counter4++; break;
            case 5: counter5++; break;
            case 6: counter6++; break;
            default: cout << "Invalid value!" << endl; break;
        }
        total++;
        cout << "Next throw: ";
```

```
        cin >> throwValue;
    }

    // Calculate and write the 6 percentages
    if (total > 0) {
        cout << "Percentage for 1: " << (counter1 * 100.0 / total) << "%" << endl;
        cout << "Percentage for 2: " << (counter2 * 100.0 / total) << "%" << endl;
        cout << "Percentage for 3: " << (counter3 * 100.0 / total) << "%" << endl;
        cout << "Percentage for 4: " << (counter4 * 100.0 / total) << "%" << endl;
        cout << "Percentage for 5: " << (counter5 * 100.0 / total) << "%" << endl;
        cout << "Percentage for 6: " << (counter6 * 100.0 / total) << "%" << endl;
    } else {
        cout << "No valid throws were made." << endl;
    }

    return 0;
}
```

Listing 8.6: Dice without array

Question 8.3
Change the dice program in Listing 8.6 to make use of an array to capture the number of times a number of a 6 sided dice has been thrown.

8.2.1 Finding the size of an array

There are two ways of determining the size of a static array, where the array is defined at compile-time, within the scope in which the array was defined. The first is by using the `sizeof` function or by using pointer arithmetic. Note, the size of an array received as a parameter or as a return type, cannot be calculated. Trying to calculate the size of a dynamic array (refer to Section 8.5) may lead to unpredictable results.

To calculate the size of an array using the `sizeof` function you first need to determine the number of bytes the array uses and then divide it by the number of bytes of the type stored in the array. For example, **int** myArraySize = **sizeof**(myArray) / **sizeof**(**int**); is used to calculate the number of integer elements in an array `myArray`.

The following code makes use of pointer arithmetic to calculate the size of the array `arr`, **int** size = ∗(&arr + 1) − arr;. The statement uses the start and end of the array for its calculations. The expression `*(&arr+1)` represents a pointer to the address of the end of the array and a pointer to the the address of the start of the array (`arr`) is subtracted from this address value. So what does `*(&arr+1)` mean? `&arr` is a pointer to the entire array. Adding 1 gives the address to just past the end of the array. Dereferencing this address, gives a pointer the the memory just after the array, from which the pointer to the first element of the array can be subtracted.

8.2.2 Sending Arrays as Parameters to Functions

When sending an array to a function as a parameter or receiving an array from a function, either a reference to the array, or a pointer to the array is passed. Arrays can be passed to functions by:

- specifying the exact size of the array the function should expect `void printArray(int values[5], int size );`
- defining an array of no specific size, thereby enabling the function to be used for array id varying sizes, `void printArray(int values[], int size );`, or
- sending a pointer to the the array, `void printArray(int *values, int size );`

For all three these cases, it is necessary to pass the size of the array through to the function from the calling function or vice-versa if the array were to be declared within the function for which it is defined as a parameter. This is because a pointer to the first element of the array is sent to the function and the calculation to determine the size of the array will simply return the size of the type of the first element of the array and not the number of elements in the array.

Iterating through the array can take place either using the index or using pointer arithmetic

```
void printArray(int *values, int size) {
    for (int i = 0; i < size; i++) {
        cout << values[i] << " ";
    }
    cout << endl;
}
```

or

```
void printArray(int values[], int size) {
    for (int i = 0; i < size; i++) {
        cout << *(values + i) << " ";
    }
    cout << endl;
}
```

8.3 One-Dimensional Array Plans

Other than defining and printing the elements in the array, we would like to initialise an array or to insert values into an array. The first 3 plans under consideration will focus on populating the array with valid values. The plans in Section 8.3.2 provide a means to sort and search arrays.

8.3.1 Plans for populating arrays with values

As with variables, arrays need to be populated with values. Many of the plans in Study Unit 6 translate well to use arrays. Plans 8.A and 8.B are examples of how Plans 6.A and

6.C respectively translate to arrays. Plan 8.C

Plan 8.A: Read known number of array elements

Hint: It would be advisable to place these statements in a separate function since reading in all values occurs once in the program only. Then you need not "clutter" the main program with extra statements, only with a statement similar to something like: `readArrayValues()`.

```
void readArrayValues(int arr[], int size) {
    for (int i = 0; i < size; ++i) {
        cin >> arr[i];
    }
}
```

Question 8.4
Read 10 integer values from the keyboard and determine the average of the values. Count the number of values below the average. Write the results to the screen.

Plan 8.B: Read unknown number of array elements

Arrays have a fixed maximum number of elements and therefore the number of unknown values that can be entered may not exceed this number. The condition to stop reading values is therefore based on both a sentinel as well as the maximum size of the array.

```
int counter = 0;
cin >> value[counter];
while ((value[counter] != 999) && (counter < maxArraySize)) {
    counter++;
    cin >> value[counter];
}
if (value[counter] == 999) {
    counter--;
}
```

In this plan, the number of elements entered into the array may not exceed `maxArraySize`. It may however be less if the sentinal of `999` is entered for value. The variable `counter` indicates how many values were entered into the array. Note the if-statemenet after the loop that adjusts counter to not incluse the sentinal value in the elements entered into the array.

Question 8.5
Would it be better to write the plan in terms of a do-while statement?

Question 8.6

1. Why is it necessary to have a double condition to stop the loop?
2. What would happen if the condition `counter < maxArraySize` is left out?
3. Why is the if-statement necessary?

Plan 8.C: Initialise array elements

All elements in the array are assigned an initial value (also advisable to do in a separate function).

```
void initializeArray(int arr[], int size, int initialValue) {
    for (int i = 0; i < size; ++i) {
        arr[i] = initialValue;
    }
}
```

Or, a shorthand which replaces the statements in the function:

```
int arrayName[sizeConstant] = {initialValue};
```

Consider the example in which the flat number of the person for which the postman has letters is read in. The number of letters in the post box is incremented by the number of letters delivered. The program prints the number of letters in each of the post boxes in the flat building. Assume that the flat building has 30 flats.

```
#include <iostream>
using namespace std;

int main() {
    int postBox[30] = {0}; //Shorthand to initialise all box numbers to 0
    int boxNumber, letterNumber;

    cout << "What is the box number? ";
    cin >> boxNumber;
    while (boxNumber != 0) {
        cin >> letterNumber;
        postBox[boxNumber - 1] += letterNumber;
        cout << "Next box number (0 to end)? ";
        cin >> boxNumber;
    }

    for (int i = 0; i < 30; ++i) {
        cout << "Flat " << i + 1 << " received " << postBox[i] << " letters" << endl;
    }

    return 0;
}
```

Listing 8.7: Post Box Example

Question 8.7
Why are the values of `postBox` initialized to `0`?

Question 8.8
Change the program so that it makes use of functions.

Question 8.9
Add functionality to the program to determine which flat got the most letters. Make use

of the Determine Highest Value plan and adapt it for arrays. Write a separate function, e.g. `getFlatWithMostLetters()`.

Question 8.10
Write a program that reads the names and marks of a maximum of 100 students. Determine the name(s) of the student with:

- The highest mark
- The lowest mark
- A mark equal to the average (NOTE: There might not be a mark exactly equal to the average)

8.3.2 Plans for Sorting and Searching Arrays

Values are typically placed sequentially in arrays (in the order in which they are entered). Keeping this ordering is not always conducive to timeous processing of the values within the arrays. Array algorithms, such as sorting and searching can help improve element access and manipulation.

In this section we will consider two types of array algorithms. We will begin with sorting as searching through an unsorted array takes time. Once an array is sorted, more advanced searching algorithms can be applied.

8.3.2.1 Sorting

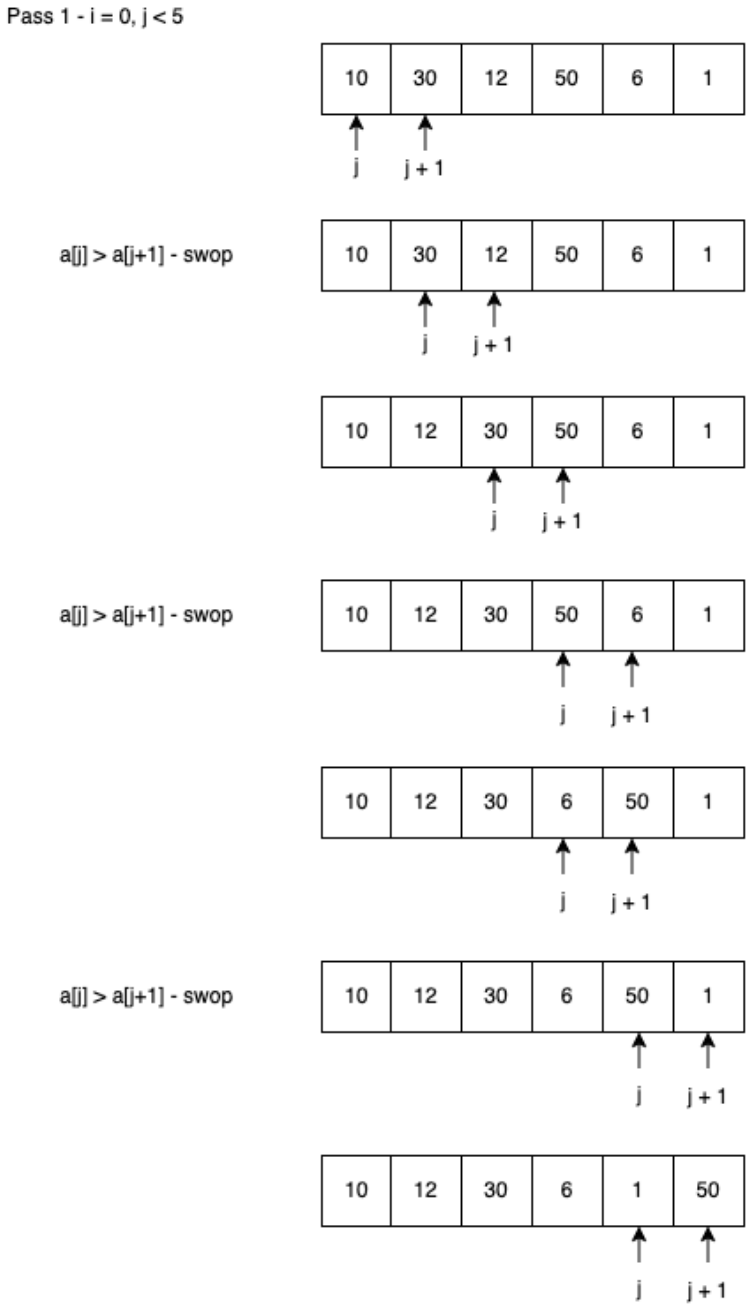
Sometimes it is necessary to order the elements in an array. This can be either in ascending or descending order. Most sorting algorithms will compare two values and then swop the values if the sorting condition does not hold for the two values under consideration. The following two plans will sort the array in ascending order using the Bubble sort algorithm and the Selection sort algorithm.

Plan 8.D: Bubble sort

Bubble sort continuously loops through the array to be sorted and compare adjacent elements. If these adjacent elements are out of order, it swops the elements. This process continues until the array is sorted.

```
void bubbleSort(int arr[], int size) {
    for (int i = 0; i < size − 1; ++i) {
        for (int j = 0; j < size − i − 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                swop(arr[j], arr[j + 1]);
            }
        }
    }
}
```

When `i = 0`, `j` will iterate over 0 through to 5 and compare the value at `arr[j]` with the one at `arr[j + 1])`. If the value of the element at position `j` is larger than the one at position `j+1`, the values are swopped and j is incremented by 1. The following figure shows how the values in the array are switched during the first pass of the algorithm (that is when `i = 0`).



As `i` iterates from 0 to 5, there will be a total of 6 passes over the array with `j` iterating over one less element than in the previous pass over subsequent passes.

Question 8.11
Draw the changes to the array and the final array after the second pass, that is for `i = 1`.

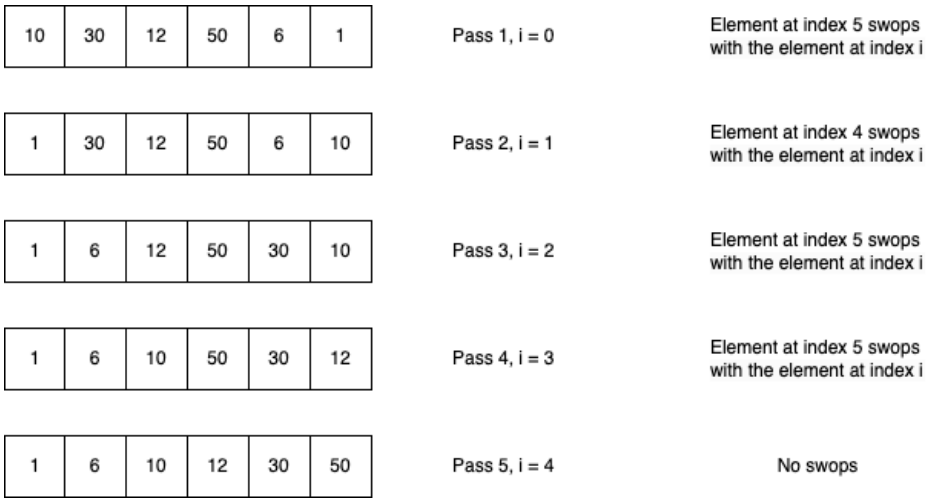
Plan 8.E: Selection sort

Selection sort selects the element containing the smallest value from the unsorted portion of the array and places the element in the correct position by swopping it out with the element currently in the position.

A function implementing the selection sort algorithm, assuming a valid swop function exists, is given below.

```
void selectionSort(int arr[], int size) {
    for (int i = 0; i < size − 1; ++i) {
        int minIndex = i;
        for (int j = i + 1; j < size; ++j) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            swop(arr[i], arr[minIndex]);
        }
    }
}
```

At the start of the first pass (where `i = 0`) of the algorithm, the first element is assumed to hold the smallest value. The loop indexed by `j` will check if there are any values smaller than this value, if there is, the index of the smallest element (`minIndex`) will be changed accordingly. If the $\texttt{i}^{th}$ element is not the smallest element, the swop function is called. For each pass `i + 1`, `j` iterates from the index of `i` to find the smallest element and calls the swop function at the end of the pass as needed. The figure below shows how the array has changed with each pass.

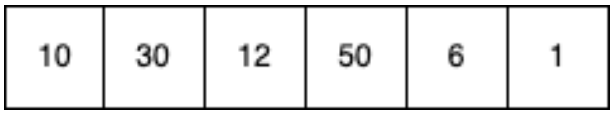


Overall, selection sort is considered more efficient. It generally requires less swops that what bubble sort does.

8.3.2.2 Searching

We make use of searching algorithms to determine whether the array contains a specific value. A value can be searched for in either an unsorted or a sorted array. Searching is however more efficient on sorted arrays.

Consider the following unsorted array and assume we are wanting to determine whether the value 6 is in the array. If we begin at the beginning of the array, it will take us 5 comparisons to determine that 6 is indeed in the array. If the array were sorted in ascending order, it would take 2 comparisons.



Now consider the scenario where the value 13 is being looked for. In the unsorted array, we would have to test every element in the array and only when we have reached the end of the array can we say that the element is not in the array. If the array was sorted in an ascending order, when we reach an element with a value that is greater than the value we are looking for, we can proclaim that the element is not in the array.

In this section we will be considering two search algorithms that can be applied to arrays. The first is the linear search, as described above, which can be applied to both unsorted and sorted arrays. The second is the binary search algorithm which must be applied to a sorted array.

Plan 8.F: Linear search

Linear search will begin at one end of the array and move from element to element of the array checking whether the value being searched for has been found.

```
int linearSearch(int arr[], int size, int searchElement) {
    for (int i = 0; i < size; ++i) {
        if (arr[i] == searchElement) {
            return true;
        }
    }
    return false;
}
```

Question 8.12
Rather than having the search algorithm return a boolean value giving an indication whether the value being searched for is in the array or not, the algorithm can be adapted to return the location in the array where the value can be found. If the value is not in the array, a `-1` can be returned. Rewrite the linear search algorithm to provide this functionality.

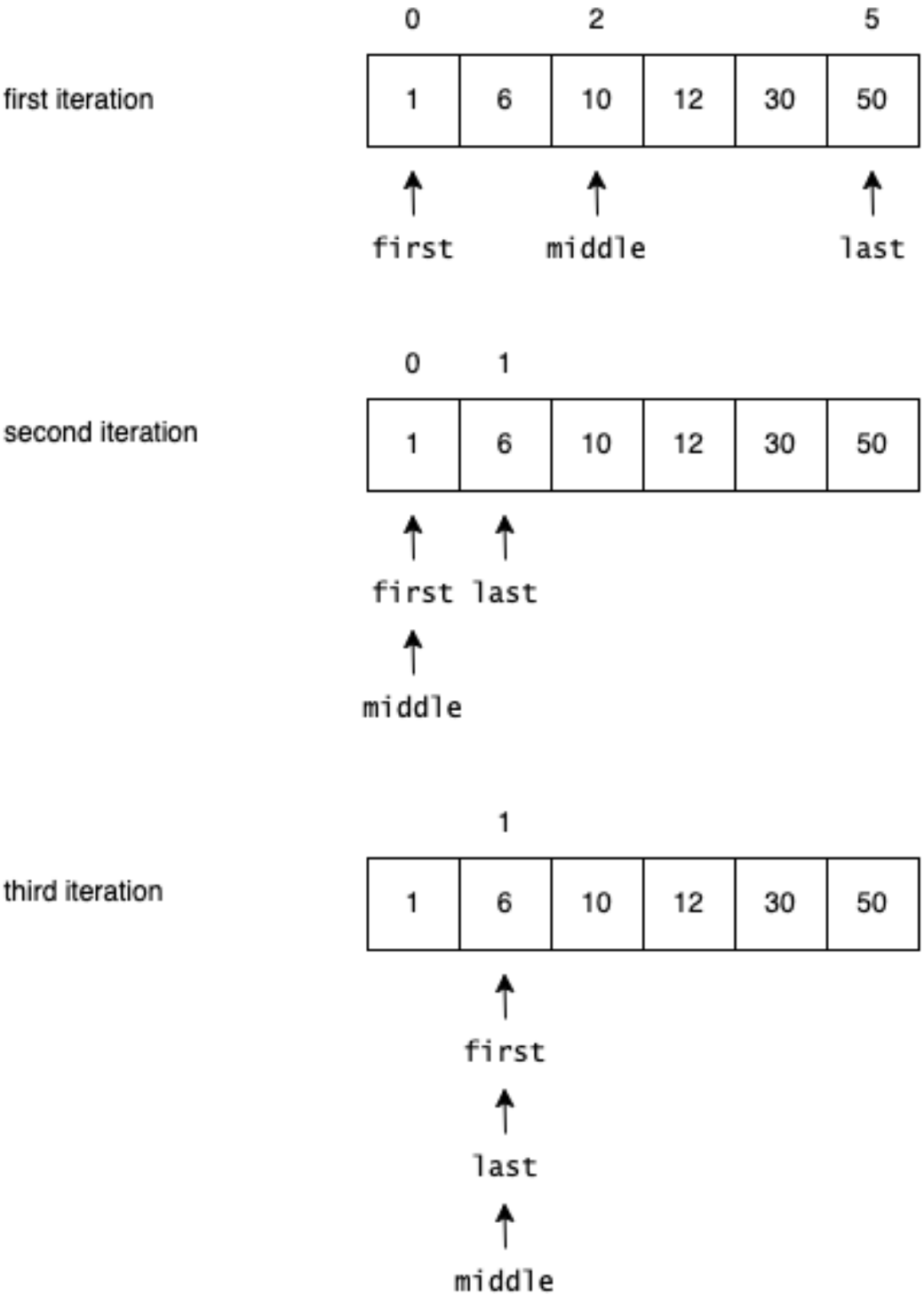
Plan 8.G: Binary search

Binary search systematically halves its search space by considering where in the array the value being looked for lies in relation to the middle element in the array. For the next

iteration of the search, the algorithm will look in the part of the array where it is most likely to find the element. If the element is not in the array, the boolean value of `false` is returned.

```
bool binarySearch(int arr[], int size, int searchElement) {
    int first = 0, last = size − 1;
    while (first <= last) {
        int middle = (first + last) / 2;
        if (arr[middle] == searchElement) {
            return true;
        } else if (arr[middle] < searchElement) {
            first = middle + 1;
        } else {
            last = middle − 1;
        }
    }
    return false;
}
```

The following trace shows how the algorithm is applied to the sorted array with values {`1, 6, 10, 12, 30, 50` } and the value being searched for as 6. For the first iteration of the while loop, the values of first and last are `0` and `size - 1` respectively. The index `middle` is calculated to be `2`. Remember, integer division is applied (`5/2 = 2`). As the value being searched for is less than the middle value, it is necessary look to the left of the middle point in the array. The index for last is adjusted and middle is recalculated. The algorothm will terminate with true, if the value being searched for is at index middle, otherwise when index first is greater than index last, it can be safely concluded that the element is not in the array.



Question 8.13
Set up your own trace to find the element with value 12 in the sorted array.

Question 8.14
Add the following functionality to your solution of Question 8.2:

- First add a function that sorts the array of marks and the array of names in ascending order. To sort the names, you simply have to swap the i and j elements of the name array whenever the marks are swapped.

- Then ask the user for a mark and determine whether there is a student that achieved the specified mark.

  Write the program twice, once using the linear search method and for a second time using the binary search method.

8.4 Multi-dimensional Static Arrays

If the elements of an array are again of type array, the structure is called a matrix or a two-dimensional array. Any data that is normally represented as a table containing values of the same type, can be stored as an array in memory. For example, the votes that candidates achieved in different voting areas is an example of data that can be stored as a matrix.

The matrix of float values, given below, comprises of 3 rows and 4 columns. As the row and column headings are not of the same type as the contents of the matrix, they will need to be stored in separate structures. These will be referred to as `area` and `candidate` respectively.

	Candidate			
Area	A	B	C	D
1	10	7.5	1	0.5
2	22	15	0.7	0
3	6.5	8	5	4

The code in Listing 8.8 reads the votes from the user and generates the necessary row and column headings when the matrix is printed to the screen. It is assumed that the votes to be capture are for exactly 3 areas and 4 candidates.

```
#include <iostream>
using namespace std;

int main() {
    float matrix[3][4];
    int area[3];
    char candidate[4];

    int i, j;

    // Assign the candidate designations
    for (j = 0; j < 4; ++j) {
        candidate[j] = 'A' + j;
    }

    // Read the values into the matrix
    for (i = 0; i < 3; ++i) {
        area[i] = i + 1; // can allocate areas automatically
```

```
        for (j = 0; j < 4; ++j) {
            cout << "Matrix[" << i + 1 << "][" << j + 1 << "]: ";
            cin >> matrix[i][j];
        }
    }

    // Write the matrix values to the screen
    cout << '\t' << "Candidate" << endl;
    cout << "Area" << '\t';
    for (j = 0; j < 4; ++j) {
        cout << candidate[j] << '\t';
    }
    cout << endl;
    for (i = 0; i < 3; ++i) {
        cout << area[i] << '\t';
        for (j = 0; j < 4; ++j) {
            cout << matrix[i][j] << '\t';
        }
        cout << endl;
    }

    return 0;
}
```

Listing 8.8: Votes Matrix Example

The loop to assign candidate designations assumes that candidates are "named" beginning with 'A' and ending with 'A'+3 or 'D'. The next set of nested loops automatically assigns the area value (1, 2 or 3) to the corresponding area array and reads 12 (3*4) values from the console and places them in the matrix for each candidate in the particular area. The final loops, write the matrix out in the given form.

Question 8.15
Why is the `cout << endl` necessary?

Question 8.16
What do lines `for (j = 0; j < 4; ++j)` do?

Question 8.17
What is the function of lines `area[i] = i + 1;`, `cout << "Area" << '\t';` and `cout << area[i] << '\t';`? What would the output be without them?

The program given in Listing 8.9 is an modified version of the program in Listing 8.8. In this version the matrix is read in using functions and the matrix and resultant arrays are passed to the functions using parameters. Note, arrays are passed to functions by reference.

```
#include <iostream>
using namespace std;

const int ROWS = 3;
const int COLS = 4;
```

```
void readMatrix(float matrix[ROWS][COLS]) {
    cout << "Type in the votes (in 1000's) for each of the following\n";
    for (int i = 0; i < ROWS; ++i) {
        for (int j = 0; j < COLS; ++j) {
            cout << "Voting area: " << i + 1 << " Candidate: " << char('A' + j) << " ---> ";
            cin >> matrix[i][j];
        }
    }
}

void sumPerCandidate(float matrix[ROWS][COLS], float totals[COLS]) {
    for (int j = 0; j < COLS; ++j) {
        totals[j] = 0;
        for (int i = 0; i < ROWS; ++i) {
            totals[j] += matrix[i][j];
        }
    }
}

void writeResults(float totals[COLS]) {
    float sum = 0;
    cout << "Candidate   Total Votes\n";
    for (int j = 0; j < COLS; ++j) {
        cout << char('A' + j) << "          " << (totals[j] * 1000) << "\n";
        sum += totals[j];
    }
    cout << "-------------------------\n";
    cout << "Total       " << (sum * 1000) << "\n";
}

int main() {
    float matrix[ROWS][COLS];
    float totals[COLS];

    readMatrix(matrix);
    sumPerCandidate(matrix, totals);
    writeResults(totals);

    return 0;
}
```

Listing 8.9: Votes Matrix Example, Version 2

The function `readMatrix` reads a fixed size matrix of rows (3) and columns (4) in. Each element in the matrix represents, the votes (in 1000's) that the candidate gained for a particular voting area. The function `sumPerCandidate` calculates the sum of the votes per area each candidate achieved. This result is placed in the totals array which is then passed back to the main function and into the `writeResults` function for displaying on

the screen. Note how the functions are called from the main function.

Question 8.18
Write a function to determine the candidate with the highest number of votes.

8.4.1 Applying C++ Matrices to Mathematics

Linear algebra and particularly matrices are prevalent in computer science. Both computer graphics and artificial intelligence make use of vectors and matrices.

When adding two matrices, both matrices must be of the same size. The formula for matrix addition is given by:

For two matrices A and B of the same dimensions $m \times n$:

$$C_{ij} = A_{ij} + B_{ij}$$

where C is the resulting matrix.

If we are to write out the result of the formula for two 3×4 matrices, it will be as follows:

```
a11 a12 a13 a14   b11 b12 b13 b14   a11+b11 a12+b12 a13+b13 a14+b14
a21 a22 a23 a24 + b21 b22 b23 b24 = a21+b21 a22+b22 a23+b23 a24+b24
a31 a32 a33 a34   b31 b32 b33 b34   a31+b31 a32+b32 a33+b33 a34+b34
```

Question 8.19
Write a program to add two matrices A and B.

Consider the formula for matrix multiplication: For matrices A of dimension $m \times n$ and B of dimension $n \times p$:

$$C_{ij} = \sum_{k=1}^{n} A_{ik} \times B_{kj}$$

where C is the resulting matrix of dimension $m \times p$.

The following program multiplies two matrices together using the above matrix multiplication formula.

```
#include <iostream>
using namespace std;

const int M = 3;
const int N = 2;

int main() {
    float matrixA[M][N] = {{1.0, 0.0}, {0.0, 2.5}, {1.5, 0.0}};
    float matrixB[N][N] = {{1.0, 1.0}, {1.0, 1.0}};
    float matrixC[M][N] = {0};

    for (int i = 0; i < M; ++i) {
        for (int j = 0; j < N; ++j) {
```

```
            for (int k = 0; k < N; ++k) {
                matrixC[i][j] += matrixA[i][k] * matrixB[k][j];
            }
        }
    }

    for (int i = 0; i < M; ++i) {
        for (int j = 0; j < N; ++j) {
            cout << matrixC[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Listing 8.10: Matrix Multiplication

Question 8.20
Currently the two programs you have written for matrix manipulation use matrices that have been defined at compile-time and hard-coded in the program. Change the program in Listing 8.10 to read the values of `matrixA` and `matrixB` from the keyboard.

8.4.2 Calculating the Size of a Two-dimensional Static Matrix

As with one-dimensional arrays, it is possible to calculate the size of a two-dimensional array. We first need to calculate the number of columns the array comprises of before we can determine the number of rows the matrix has. We do the calculation as follows:

- We can calculate the number of columns in a row by:
  int cols = **sizeof**(matrix[0])/**sizeof**(**float**);
- Calculating the number of rows requires us to know the number of columns as well as the size of a float:
  int rows = **sizeof**(matrix)/**sizeof**(**float**)/cols;

The same restrictions apply. That is, you can only apply the calculations in the function in which the array has been defined.

8.4.3 Functions and Matrices

Rather than having individual programs for each of the matrix manipulation functions, as in Listing 8.10. It would be better to have a library of matrix manipulation functions. To do this we need to define functions that will accept and return matrices as parameters. When we send (or receive) a static two-dimensional array to (or from) a function as a parameter, we need to define the number of columns the matrix comprises of as a parameter of the function.

As with one-dimensional static arrays, we need to send the number of rows and columns as parameters along with the matrix.

Consider the following function to print the values of a given matrix to the console.

```
void printMatrix(float m[][4], int rows, int cols) {
    for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                    cout << fixed << setprecision(1)
                         << m[i][j] << " ";
            }
            cout << endl;
    }
}
```

Listing 8.11: Printing values into a static 2D matrix

If we define the function in Lisitng 8.11 using the pointer notation, the number of columns still needs to be specified as follows:

```
void printMatrix(float (*m)[4], int r, int c);
```

Question 8.21
How does the implementation of the `printMatrix` function change when using the pointer notation definition?

Question 8.22
Write a function to read values from the keyboard into the matrix.

As with one-dimensional arrays, two-dimensional arrays can be stepped though using pointer arithmetic instead of indices. Consider the following code showing the function given in Listing 8.11 using pointer arithmetic.

```
void printMatrixPtrArith(float m[][4], int r, int c) {
    for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                    cout << fixed << setprecision(1)
                         << *(*(m+i)+j) << '\t';
            }
            cout << endl;
    }
}
```

Note how the row increment, `*(m+i)`, interacts with the column increment, `*(*(m+i)+j)`, where `i` is the row increment and `j` the column increment.

The matrix however is limited to 4 columns and any number of rows when passed as a parameter. If we do not use part of the array, for example, if we wish to print a 2×2 matrix, we will need to send a matrix that has two rows and four columns to the function. This effectively wastes space on the stack of $(2 * 2 * \text{sizeof(float)})$ bytes on the stack. Alternatively, if we want to use the function to print a 10×10 matrix, we cannot do so with this function. The number of columns of 10 exceeds the number of columns (4) expected by the function.

This problem is as a result of 2D arrays being placed in stack memory in contiguous memory locations. In order to calculate where the next row of a 2D array begins, we need to know how many columns we have. Each row of a 2D array placed on the stack therefore has exactly the same number of columns.

To solve the problem of a fixed number of columns when sending a static array as a parameter to a function, we need to consider placing the arrays on the heap. Arrays on the heap (dynamic arrays) are defined as an array of pointers to arrays. Section 8.5 will introduce dynamic arrays.

8.5 Dynamic Arrays

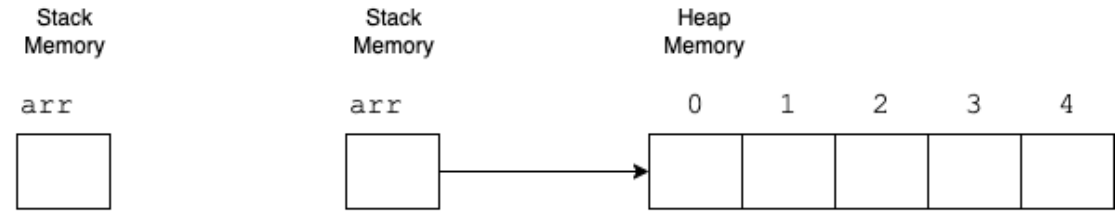
Limitations of static one-dimensional and two-dimensional arrays are that they are fixed at compile time in terms of their size. The maximum possible size of the structure needs to be reserved at compile-time and cannot be changed at run-time when the actual size of the structure required becomes known. Dynamic arrays, are a run-time array in both their one-dimensional or multi-dimensional versions. The memory required for the array structure is allocated at run-time and is therefore allocated on the heap.

8.5.1 One-dimensional dynamic arrays

To define a one-dimensional dynamic array, the following statement needs to be used:

type ∗arr;
arr = **new** type [size];
// or all in one line, type ∗arr = new type [size];

where `size` can be a constant or entered by the user. The dynamic array variable resides on the stack and the array itself is on the heap in contiguous memory locations of `int` size. The figure below illustrates how an `int` array of size 5 is created on heap memory.




```
int *arr;          arr = new int[5];
```

Remember, if we allocate memory on the heap, we need to delete the memory from the heap when we no longer need it. To do so for a dynamic array we use the `delete` operation in the following statement for the given example: **delete** [] arr;.

The following program illustrates defining, using and deleting a dynamic one-dimensional array where the user specifies the size of the array.

```
#include <iostream>
using namespace std;

int main() {
    int *arr;
    int size;

    cout << "How many elements would you like to store? ";
    cin >> size;
    arr = new int[size];

    cout << "Please enter your " << size << "values" << endl;
    for (int i = 0; i < size; i++) {
        cout << i + 1 << ": "; cin >> arr[i];
    }

    cout << "The values entered are: ";
    for (int i = 0; i < size; i++) { cout << arr[i] << " "; }
    cout << endl;

    delete [] arr;
    return 0;
}
```

Listing 8.12: One-dimensional dynamic array example

8.5.1.1 One-dimensional Dynamic Arrays as Parameters to Functions

Sending a dynamic one-dimensional array to a function is no different from sending a one-dimensional static array to a function. The `printArray` functions with parameters `int arr[]` and `int *arr` defined in Section 8.2.2 will work as is for dynamic one-dimensional arrays. As with static arrays, errors will occur if the function with parameter `int arr[4]` is used and the bounds of the array is exceeded.

This means that the binary search algorithm given in Plan 8. G will work as is when called using the dynamic array `arr` as defined in Listing 8.12

8.5.2 Two-dimensional dynamic arrays

Two-dimensional arrays become a little more complex. They are defined using a one-dimensional array on the heap representing the rows of the two-dimensional dynamic array. Each of the elements in this array is a pointer to another one dimensional array. Figure 8.1 illustrates a two-dimensional dynamic array.

8.5.2.1 Variable number of columns per row

Each row in the two-dimensional dynamic array does not have to have the same number of columns. This makes the array difficult to manage. There are two possible solutions

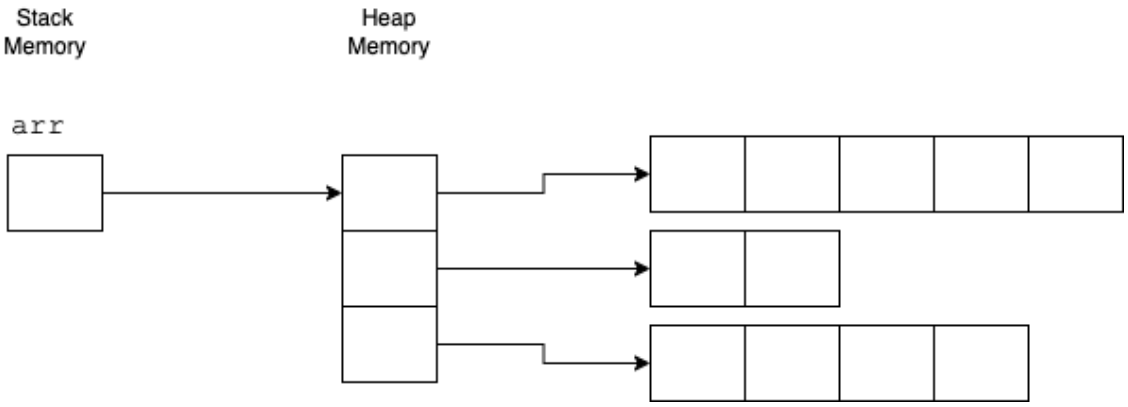


Figure 8.1: Two-dimensional Variable Size Array

to overcome this hurdle. The first is to manage a parallel array that stores an integer value indicating the number of columns each respective row has. The second solution is to merge this integer value and the pointer to the row elements into a C++ structure called a `struct`. C++ `struct`s will be covered in the following study unit.

The following code is required to manage the two-dimensional dynamic array structure given in Figure 8.1 above. We assume the value for `size` (3 in this example) is entered by the user before the arrays are declared and created. The user is also asked to enter the number of elements for each row.

```
int **arr;
int *parallelArr;

arr = new int*[size];
parallelArr = new int[size];
for (int i = 0; i < size; i++) {
    cin >> parallelArr[i]; // number of columns in row
    arr[i] = new int[parallelArr[i]];
}
```

`arr` is a pointer to an element, in this case represented as an array, that contains pointers to another array holding integer values. To create the first dynamic array, which holds the pointers to each row, we create an array with `size` elements of type `int*` in. For each of the elements in this array, indexed from 0 to `size-1`, we create another dynamic one-dimension array representing each row of the matrix. The user is asked for the sizes of each of these rows and will enter the value into the parallel array (`parallelArr`) for the specific row index. For the given example the values are 5, 2 and 4 for array indexes 0, 1 and 2 respectively. `parallelArr` is a normal dynamic one-dimensional dynamic array. Once this structure has been created, it can be used.

A function to print the array given in Figure 8.1 is given in the listing below:

```
void printArray(int **a, int *pa, int size) {
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < pa[i]; j++) {
            cout << a[i][j] << "\t";
        }
        cout << endl;
```

```
    }
}
```

This function takes two array parameters, the first is the two-dimensional array structure defined as `int **a`, and the second the one dimensional parallel array, `pa`, that holds the number of elements in each row.

These structures, as with all memory allocated on the heap, need to be deleted from the heap. The following code will do so.

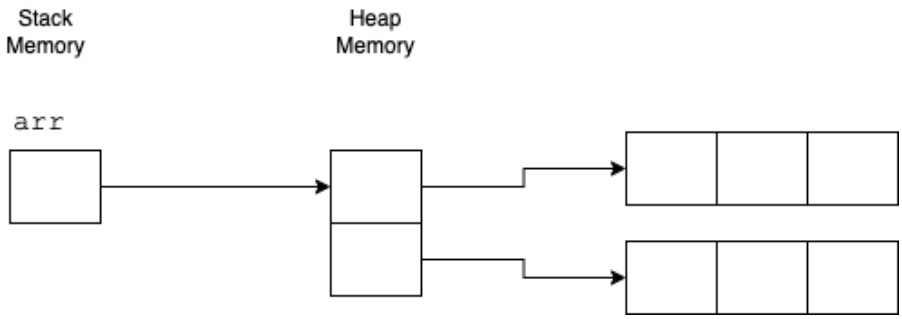
```
delete [] parallelArr;
parallelArr = NULL;
for (int i = 0; i < size; i++) {
    delete [] arr[i];
}
delete [] arr;
arr = NULL;
```

For the two-dimensional dynamic array structure, the arrays representing the column values are deleted first, before the dynamic array with pointers to these structures.

8.5.2.2 Fixed number of columns per row

Two-dimensional dynamic arrays with a fixed number of columns are easier to manage. These are akin to the static matrix we considered in Section 8.4.

As an example, assume we want to create a 2×3 matrix using two-dimensional dynamic arrays as in the figure below:



To create the matrix, we complete the following steps:

- We make use of double pointers to define the initial variable on the stack and initialise the variables that represent the size of the matrix:

  float $**$arr;
  int rows = 2, cols = 3;

- Create the array representing the 2 rows:

  arr = **new float**$*$[rows];

- Create 2 arrays with 3 elements and link them to the rows:

```
for (int i = 0; i < rows; i++) {
    arr[i] = new float[cols];
}
```

We now have a dynamic 2×3 matrix that we can use. Remember to delete the matrix before going out of scope.

Question 8.23
Write the code to read values into the matrix and delete the matrix from heap memory.

8.5.2.3 Creating a Matrix Library

In Section 8.4.1 writing code to manipulate, particularly adding and multiply, matrices was considered. Mention was made of this being restrictive due to the column parameter needing to be constant. For example, the definition for a function to add two matrices would be given by:

```
void addMatrices(float a[][4], float b[][4], float result[][4],int rows, int cols);
```

and thereby allowing a maximum of 4 column matrices to be manipulated. With dynamic matrices, this restriction no longer exists. We can now define the function as follows:

```
void addMatrices(float **a, float **b, float **result, int rows, int cols);
```

or even better:

```
float** addMatrices(float **a, float **b, int rows, int cols);
```

The code for the latter definition is given by:

```
float** addMatrices(float **a, float **b, int rows, int cols) {
    float** matrix = allocateMatrix(rows, cols);

    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            matrix[i][j] = a[i][j] + b[i][j];
        }
    }

    return matrix;
}
```

Question 8.24
Write the code to multiply two matrices. The header for the function is given by:

```
float** multiplyMatrices(float** A, int rowsA, int colsA, float** B, int rowsB, int colsB)
```

Remember the constraint that the number of columns of matrix `A` must equal the number of rows of matrix `B` and the resultant matrix is (number of rows of A $\times$ `number of columns of` B).

You can now begin creating a library of functions for the manipulation of matrices. The header file (`Matrix.h`) for this library is given by:

```
#ifndef MATRIX_H
#define MATRIX_H

        float** allocateMatrix(int, int);
        void deallocateMatrix(float**, int);

        void readMatrix(float**, int, int);
        void printMatrix(float**, int, int);

        float** addMatrices(float**, float**, int, int);
        float** multiplyMatrices(float**, int, int, float**, int, int);

#endif
```

Question 8.25
Add the implementation for all these functions to a file called `Matrix.cpp`.

Question 8.26
Test your matrix manipulation library with the following main program.

```
#include <iostream>

#include "Matrix.h"

using namespace std;

int main() {

        float **matrixA, **matrixB, **matrixC;

        matrixA = allocateMatrix(2, 3);
        matrixB = allocateMatrix(2, 3);
        matrixC = allocateMatrix(3, 4);

        cout << "Type in the matrix A:" << endl;
        readMatrix(matrixA,2,3);

        cout << "Type in the matrix B:" << endl;
        readMatrix(matrixB,2,3);

        cout << "Type in the matrix C:" << endl;
        readMatrix(matrixC,3,4);



        cout << "A = " << endl;
        printMatrix(matrixA,2,3);
        cout << endl << "B = " << endl;
        printMatrix(matrixB,2,3);
        cout << endl << "C = " << endl;
```

```
        printMatrix(matrixC,3,4);
        cout << endl;

        float** addResult = addMatrices(matrixA,matrixB,2,3);

        cout << "A + B = " << endl;
        printMatrix(addResult,2,3);

        float** multiplyResult = multiplyMatrices(matrixA,2,3,matrixC,3,4);

        if (multiplyResult != NULL) {
                cout << "A * C = " << endl;
                printMatrix(multiplyResult,2,4);
        }

        if (multiplyResult != NULL) deallocateMatrix(multiplyResult,2);
        deallocateMatrix(addResult,2);
        deallocateMatrix(matrixC, 3);
        deallocateMatrix(matrixB, 2);
        deallocateMatrix(matrixA, 2);

    return 0;
}
```

Note the order of deallocation of the dynamic memory is the reverse over od allocation. This is a C++ requiremnent.

8.6 Document Change Log

Date	Change
17 June 2024	Initial preparation and presentation of the document.